

ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ ΕΡΓΑΣΙΑ 1(p3220273,p3210271)

Για το μέρος A, δημιουργήσαμε την κλάση `StringDoubleEndedQueueImpl` η οποία είναι διεπαφή της `StringDoubleEndedQueue` η οποία χειρίζεται μια διπλά συνδεδεμένη ουρά. Μέσα σε αυτήν υπάρχει η `private class node`, ο `constructor` της κλάσης και οι παρακάτω μέθοδοι της κλάσης : `isEmpty, addFirst, addLast, RemoveFirst, RemoveLast, getFirst, getLast, size, printQueue`. Η `Node` μας βοηθάει να αναπαραστήσουμε τα στοιχεία της ουράς, το `prev` δείχνει την διεύθυνση του προηγούμενου στοιχείου της ουράς(αν δεν υπάρχει είναι `null`), το `item` είναι το στοιχείο της ουράς και το `next` δείχνει την διεύθυνση της του επομένου στοιχείου της ουράς(αν δεν υπάρχει είναι `null`). Το `head` να δείχνει την αρχή της ουράς και με το `tail` το τέλος της ουράς. Οι μέθοδοι που επιτρέπουν την εισαγωγή και την εξαγωγή δεδομένων στην ουρά (`addFirst, removeFirst, addLast, removeLast`) εκτελούνται σε χρόνο $O(1)$ καθώς αλαλάζουν μόνο το `head` και το `tail` της λίστας(δηλαδή τους `pointers` της αρχής και του τέλους της λίστας) Έτσι, ώστε να μην χρειάζεται να ψάξουμε όλη την λίστα. η μέθοδος `printQueue` προφανώς εκτελείται σε $O(n)$ καθώς επιστρέφει το πλήθος των στοιχείων που υπάρχει στην ουρά που αυτό το κάνει κάνοντας περιήγηση σε όλη την ουρά. Η μέθοδος `size` είναι σε χρόνο $O(1)$ γιατί άπλα επιστρέφει την τιμή από το πεδίο `size`. Η `isEmpty` επιστρέφει εάν η ουρά είναι άδεια ή όχι. Οι `getters`(`getFirst, getLast`)εκτελούνται σε $O(1)$ καθώς περνούν ένα στοιχείο της ουρά είτε από την αρχή είτε από το τέλος. Σε περίπτωση που η ουρά είναι άδεια και καλέσουμε οποιαδήποτε μέθοδο εξαγωγής ή `getters` στοιχείων μας πετάει `exception: NoSuchElementException()`.

Για το μέρος B, δημιουργήσαμε την κλάση `PrefixToInfix` που μέσα έχει την συνάρτηση `convertToInfix` η οποία χειρίζεται μια διπλά συνδεδεμένη ουρά με σκοπό να την μετατρέψει μια αριθμητική έκφραση η οποία εισάγεται από τον χρήστη σε προθεματική μορφή `prefix` και μετατρέπεται σε ενδιάμεση μορφή `infix`. Η υλοποίηση λειτουργεί ως εξής: αφού ο χρήστης εισάγει την προθεματική με κενά ενδιάμεσα σε μορφή `string` αυτά διασπώνται με τη `split` βάση τα κενά των χαρακτήρων και εισάγονται σε ένα `string` πίνακα. Έπειτα η συνάρτηση `convertToInfix` χρησιμοποιεί την διπλά συνδεδεμένη ουρά για να διαβάσει την προθεματική μορφή ανάποδα. Σε περίπτωση που διαβάσει έναν τελεστή χρησιμοποιώντας την μέθοδο `removeFirst` από το πρώτο μέρος για το πρώτο και το δεύτερο στοιχείο της ουράς και αφαιρεί του δυο τελευταίους αριθμούς του πίνακα(αφού διαβάσει ανάποδα τον πίνακα) και τους βάζει με ένα τελεστή μέσα σε παρένθεσης με την βοήθεια της μεθόδου `addFirst` από το A μέρος το αποτέλεσμα να τοποθετείτε πάλι μέσα στην ουρά αυτή η διαδικασία επαναλαμβάνεται μέχρι να επεξεργαστεί όλη η προθεματική αριθμητική έκφραση. Στο τέλος με την βοήθεια της μεθόδου `size` του A μέρους ελέγχουμε αν η `infix` έκφραση είναι έγκυρη. Αν το `size` είναι 1 τότε είναι έγκυρη και μας επιστρέφει το αποτέλεσμα σε `infix`, αν είναι διάφορο αυτού τότε δεν είναι έγκυρη η έκφραση και μας εκτυπώνει μήνυμα λάθους.

Παράδειγμα: πληκτρολογούμε ως είσοδο `" + * 5 7 8 "` η οποία προθεματική αριθμητική έκφραση είναι έγκυρη. Ψάχνοντας ανάποδα, βρίσκει το πρώτο ζευγάρι πράξεων το `value "*"` και `n1` το 5 και `n2` το 8, τα κάνει `string` βάζοντας παρενθέσεις και το `value` αναμεσά στο `n1` και `n2` και το `string` είναι το `( 5 * 7 )`, το οποίο εκχωρείτε στην αρχή της σειράς. Έπειτα βρίσκει το `value "+"` και τον αριθμό 8 που τον βάζει στο `n2` και την αριθμητική έκφραση που είχαμε προηγούμενος `( 5 * 7 )` στο `n1` τα παίρνει αυτά και τα βάζει σε `string` μορφή όπως και πάνω και μας βγάζει το αποτέλεσμα `(( 5 * 7 ) + 8 )` το οποίο με την `addFirst` το βάζει στην αρχή της λίστας και η επανάληψη τελειώνει. Έπειτα το ελέγχει με την συνθήκη `s.size() == 1`

Που αν είναι αληθής επιστρέφει το πρώτο στοιχείο της ουράς που είναι το

$(5 * 7) + 8$) ή αλλιώς μας πετάει σχετικό μήνυμα λάθους. Στην περίπτωση μας η συνθήκη είναι αληθής και μας επιστρέφει $(5 * 7) + 8$ και έτσι εκτυπώνεται η infix μορφή και αν εκτελέσουμε το πρόγραμμα μας εμφανίζει το ακόλουθο αποτέλεσμα.

```
C:\Users\user\Desktop\DD>java PrefixToInfix
Enter an arithmetic expression in prefix form (please leave a space between each symbol) : + * 5 7 8
Your arithmetic expression in Infix form is : ( ( 5 * 7 ) + 8 )
```

Για το μέρος Γ δημιουργήσαμε την κλάση DNAPalindrome που μέσα έχει την συνάρτηση DNACheck η οποία χειρίζεται μια διπλά συνδεδεμένη λίστα με σκοπό να βρει αν το string που έδωσε ο χρήστης αναπαριστά DNA ακολουθία και αποφασίζει αν είναι Watson-Crick complemented palindrome. Η υλοποίηση λειτουργεί ως εξής: αφού ο χρήστης εισάγει το string, το string εισάγεται ως είσοδος στη μέθοδο DNACheck και το string DNA έπειτα κάθε σύμβολο του εκχωρείται σε θέσεις σε πίνακα ch που είναι μεταβλητή character και επιτόπου ο πίνακας ch εκχωρείται στο πίνακα ADNAlist ο οποίος είναι τύπου string. Έπειτα στην ουρά p που είναι τύπου StringDoubleEndedQueue εισάγονται με την βοήθεια της μεθόδου addLast που φτιάξαμε από το πρώτο μέρος, εισχωρούμε από την αρχή προς το τέλος του πίνακα ADNAlist μέχρι να φτάσει η λίστα στο τέλος της. Αν η ουρά μας είναι περιττός αριθμός τότε τερματίζει το πρόγραμμα τυπώνει μήνυμα σφάλματος λέγοντας μας ότι το DNA δεν είναι Watson-Crick complemented palindrome με αιτιολογία ότι ο αριθμός των νουκλεοτιδίων που εισάχθηκαν είναι περιττός σε αντίθετη περίπτωση αρχικοποιεί το κεφάλι της ουράς και το τέλος της ουράς σε 0 και ADNAlist.length-1 αντίστοιχα. Έπειτα σαρώνει την ουρά ελέγχοντας αν το head της ουράς και το tail της ουράς είναι συμπληρώματα μεταξύ τους. Σε περίπτωση που ισχύει το παραπάνω, μειώνεται η αρχή και το τέλος της ουράς p κατά ADNAlist.length θέσεις αντίστοιχα και έπειτα αυξάνεται κατά 1 το head και μειώνεται κατά ένα το tail της ουράς ADNAlist. Στην συνέχεια με την βοήθεια της μεθόδου size από το πρώτο ερώτημα ελέγχουμε το μέγεθος της ουράς p, στην περίπτωση που ισούται με 0 τότε το DNA μας είναι Watson-Crick παλινδρόμηση τυπώνοντας μας αντίστοιχο μήνυμα. Σε αντίθετη περίπτωση δεν είναι τυπώνοντας μας επίσης αντίστοιχο μήνυμα και λόγο του τρόπου υλοποίησης του προγράμματος ξέρουμε ότι αυτό ισχύει είτε λόγο λάθος χαρακτήρων, είτε κενών είτε απλά δεν είναι Watson-Crick complemented palindrome. Σε οποιαδήποτε περίπτωση από τις δύο προηγούμενες, το πρόγραμμα μετά τερματίζει.

Παράδειγμα: Έστω εισάγουμε το την ακολουθία χαρακτήρων που αναπαριστά την DNA ακολουθία "AATT". έπειτα στην μέθοδο DNACheck σπάμε το κάθε σύμβολο του string μας στο πίνακα ch που είναι μεταβλητή character και επιτόπου εισχωρούμε αυτόν τον πίνακα σε έναν άλλο πίνακα στον ADNAlist. Μετά χρησιμοποιώντας την addLast που φτιάξαμε στο πρώτο μέρος, έτσι ώστε να βάλουμε τα στοιχεία του ADNAlist στη ουρά p. Στο παράδειγμα μας γνωρίζουμε πως ο αριθμός των DNA συμβόλων είναι άρτιος αριθμός και άρα μπαίνουμε στην else του ADNAlist.length%2==1, αρχικοποιεί το head σε 0 και το tail σε ADNAlist.length-1 (στην περίπτωση μας η ούρα είναι 4 θέσεων και άρα το tail θα είναι 3). Στην συνέχεια ελέγχει αν το head είναι συμπλήρωμα του tail, στην περίπτωση μας είναι καθώς πράγματι το "A" είναι συμπλήρωμα του "T" και στην συνέχεια χάρη τις μεθόδους removeFirst() και removeLast() από το πρώτο ερώτημα, μικραίνει την queue του p το κεφάλι κατά 4 και την ουρά του κατά 4 αντίστοιχος και το head μεγαλώνει κατά 1 και το tail μικραίνει κατά 1. Αυτή η διαδικασία επαναλαμβάνεται ακόμα μια φορά πλέον

το head είναι 1 και το tail 2, αφού πάλι το “A” είναι συμπλήρωμα του “T” τότε επαναλαμβάνοντε ακριβώς οι ίδιες εντολές με πριν και τερματίζει ο βρόχος. Στην συνέχεια χάρης την μέθοδο size() από το πρώτο ερώτημα ελέγχουμε αν η ουρά p είναι άδεια ή όχι. Στην περίπτωση μας είναι και άρα μπαίνει στην πρώτη συνθήκη και τυπώνει μήνυμα ότι είναι Watson-Crick complemented palindrome.

```
AATT is a wotson-crick complemented palindrome
```